

A Chiffres Solver Algorithm

Bernd Michaely

July 8, 2025

Abstract

This document describes an algorithm to solve the *Chiffres* problem (that is, the *Chiffres* part of the game *Des Chiffres et des Lettres*).

DRAFT

1 The Problem

See github.com/berndmichaely/chiffres-solver for a short introduction to the problem and a software implementation in Java/JavaFX.

See also: fr.wikipedia.org/wiki/Des_chiffres_et_des_lettres.

2 The Algorithm

2.1 Initial observations

We consider the 4 basic arithmetic operation $+$ (*add*), $-$ (*sub*), $*$ (*mul*) and $/$ (*div*) over $\mathbb{N} = \{1, 2, 3, \dots\}$.

First, let's observe, that all the 4 operations are binary, and two of them are commutative, $(a + b) = (b + a)$ and $(a * b) = (b * a)$, and the others both are not.

That is, for *add* and *mul*, we need to consider only one of the variants, and can freely choose, which one.

For *sub*, we *can* only consider at most one of the variants, because we do not deal with negative numbers and exclude the case $a - a = 0$.

For *div*, we also can or need only consider at most one of the variants, because we do not deal with fractional numbers, and in the case of $1/1 = 1$, again one of the variants will suffice.

To sum up, we can in all cases use at most one of the two pairs $(a \circ b)$ and $(b \circ a)$ and *sort* the operands. That is, when iterating over pairs of operands, instead of using pairs $(a \circ b)$ directly, we calculate with sorted pairs $(a' \circ b')$, where $a' := \max(a, b)$ and $b' := \min(a, b)$.

As can be seen in Table 1, it will be sufficient to iterate over the elements above the diagonal only (or below, if you prefer).

2.2 A simple algorithm

Now, we want to calculate all solutions for our given problem, and the first intention is, that we will use a recursive algorithm. That is, we pick any pair of operands, apply one of the available operators $\circ \in \{add, sub, mul, div\}$, and then one of two things will happen:

- the result is the one, we were searching for $\rightarrow$ add the result together with the recursion path to the set of solutions
- otherwise replace the chosen operands with the result and repeat the above step recursively, until only one operand is left.

Operand-Index	b_1	b_2	b_3	b_4	b_5	b_6	$\dots$
a_1	$\ddots$	$(a'_1 \circ b'_2)$	$(a'_1 \circ b'_3)$	$(a'_1 \circ b'_4)$	$(a'_1 \circ b'_5)$	$(a'_1 \circ b'_6)$	
a_2		$\ddots$	$(a'_2 \circ b'_3)$	$(a'_2 \circ b'_4)$	$(a'_2 \circ b'_5)$	$(a'_2 \circ b'_6)$	
a_3			$\ddots$	$(a'_3 \circ b'_4)$	$(a'_3 \circ b'_5)$	$(a'_3 \circ b'_6)$	
a_4				$\ddots$	$(a'_4 \circ b'_5)$	$(a'_4 \circ b'_6)$	
a_5					$\ddots$	$(a'_5 \circ b'_6)$	
a_6						$\ddots$	
$\vdots$							$\ddots$

Table 1: Operand iteration and sorting

After finishing recursion, we will have found all syntactical constructs which, evaluated by arithmetic semantics, will provide a solution for our given problem. So great, we are done! Or aren't we? Let's take a closer look at the results.

2.3 Redundancies

Let's assume the following problem: $(1, 4, 4, 5, 5, 20)$ to calculate 101. Let's apply our algorithm and have look at the results. At first, we are happy, because it produces a lot of results, see Table 2.

But a closer look shows, that we got a lot of redundancies. What is the reason?

2.3.1 Duplicate values

First, we have given the same operand values several times. Because the algorithm uses indexed access to an input array (e.g. $op[1]$ and $op[2]$ instead of the value 4 and $op[3]$ and $op[4]$ instead of the value 5), the same values are used repeatedly as often, as they appear in the input, as we can see in the first block of Table 2. Even worse, such duplicate values can appear in addition as intermediate results during the calculation.

2.3.2 Permutations of operations

In the second block of Table 2, there are solutions, which only differ in the order of their operations, but the operations themselves are the same.

1. operation	2. operation	3. operation	4. operation	5. operation
$4 * 5 = 20$	$20 * 5 = 100$	$100 + 1 = 101$		
$4 * 5 = 20$	$20 * 5 = 100$	$100 + 1 = 101$		
$4 * 5 = 20$	$20 * 5 = 100$	$100 + 1 = 101$		
$4 * 5 = 20$	$20 * 5 = 100$	$100 + 1 = 101$		
...				
$4 * 5 = 20$	$20 * 5 = 100$	$5 - 4 = 1$	$100 + 1 = 101$	
$4 * 5 = 20$	$5 - 4 = 1$	$20 * 5 = 100$	$100 + 1 = 101$	
$5 - 4 = 1$	$4 * 5 = 20$	$20 * 5 = 100$	$100 + 1 = 101$	
...				
$4 * 5 = 20$	$20 * 5 = 100$	$100 + 4 = 104$	$104 - 4 = 100$	$100 + 1 = 101$
$4 * 5 = 20$	$20 * 5 = 100$	$100 - 4 = 96$	$96 + 4 = 100$	$100 + 1 = 101$
$4 * 5 = 20$	$20 * 5 = 100$	$100 * 4 = 400$	$400/4 = 100$	$100 + 1 = 101$
$4 * 5 = 20$	$20 * 5 = 100$	$100/4 = 25$	$25 * 4 = 100$	$100 + 1 = 101$
...				

Table 2: Solution set with many redundancies for $\{(1, 4, 4, 5, 5, 20), 101\}$

2.3.3 Inner loops in calculations

In the third block of Table 2, some solution paths contain loops, which return to the same intermediate value again. We would not consider this to be a *new* solution found, but rather a redundancy, which we want to be filtered out.

Think of a route planner, which will present you a way from A over B to C, and then an alternative route from A over B and D and E back to B and C. You wouldn't really appreciate that, would you?

2.3.4 Redundant single operations

There is also a fourth source of redundancy: every time we have a solution containing $2+2=4$, there will always be a similar solution containing $2*2=4$. But we can also say, that e.g for $\{(1, 2, 4), 7\}$, we always find $1+2+4=7$, and also $2*4-1=7$. So when does it stop to be trivial? We can decide to ignore this entirely, or limit the filtering to the first mentioned very simple case.

Note moreover, that the cases $1*1=1$ and $1/1=1$ can be considered as single step inner loops.

So it seems, we have to tune our algorithm quite a bit. We have sown and harvested all syntactical solutions, and now, to get the semantically valuable results, we have to separate the wheat from the chaff in a second step.

3 Improving the algorithm

Let's address the duplicate values and permutations of operations first. Since different solutions with equal values can be handled via an identity permutation, the first problem is a special case of the second.

3.1 Eliminating permutations of operations

A *solution* can be seen as a sequence of *operations* $\in \mathbb{N} \times \{+, -, *, /\} \times \mathbb{N}$.

If and only if two solutions contain exactly the same operations, but in (possibly) different order, the sequences are equal as a multi-set, and there is a permutation between the sequences.

Let's define a relation $s_1 \sim s_2$ between two solutions such that two solutions are related if and only if they equal as a multi set (or there exists a permutation between them, if you prefer). Then all similar solutions will be partitioned into the same equivalence class, and we can replace all elements of each equivalence class by a canonical representation of that partition.

That is, our algorithm will keep a mapping between a key set of representations of equivalence classes associated with a canonical element.

An equivalence class can be represented by a multi-set of the operations of a solution, or the sorted sequence of operations, if we define a linear ordering.

What can be a canonical element? We can define a linear ordering on whole solutions also, and we can choose e.g. the smallest element.

That is, the algorithm will consider a new solution candidate found, determine its equivalence class, search it in the mapping and:

- if not found, create a new mapping of this equivalence class as key and the solution candidate as value, or
- if found, compare the existing with the new solution and replace the existing with the new one, if the new one is smaller, and leave the existing one otherwise.

3.1.1 Linear orderings

It is desirable for several reasons to define a linear ordering on the basic data types, e.g. it allows an easier comparison of the final results of different implementations of the algorithm, and it allows for sorting and use of sorted data structures. A linear ordering on *solutions* can be defined as follows:

- shorter solutions are smaller than larger ones

- on equal length, the first operations are compared in second order, the second operations in third order, and so on.

A linear ordering on *operations* can be defined as follows:

1. compare the operator in first order
2. compare the first operand in second order
3. compare the second operand in third order

3.2 Eliminating inner loops

If a solution candidate contains some inner loops, that means, it should be possible to construct a shorter solution given the existing elements of the sequence. That is, the idea of the algorithm is as follows:

- Loop over all elements of the sequence of length n and construct a new sequence of length $n - 1$ which has the chosen element removed. Don't forget to reconsider any initially given operands, which had been bound by the dropped operation.
- For each loop cycle, try to construct a new solution by ignoring the original topology of the derivation of the calculation, and instead reconnect the operations by their values recursively, starting with the final result on top of the stack. If it is possible to construct a new, shorter solution with one original operation removed, this solution candidate can be marked as redundant and e.g. simply be dropped.

Note, that the outer loop can be parallelized.

3.2.1 Example

Consider the last example in Table 2. Remove the 3. or 4. operation. Start with the final operation providing the result of 101. Its operands are 1 (which is given as an initial operand) and 100. Then let's search for another operation, which provides 100 as a result. We find the 2. operation, which needs 20 and 5 as operands. 5 is again given as an initial operand, and finally the 1. operation will complete a new solution with its result of 20. We managed to construct a new solution with just a subset of the given operations and can therefore mark this solution candidate as redundant.